COLLEGE OF
INNOVATION & TECHNOLOGY

Algorithm and Complexity Analysis

CSC-379-01 / CSC-575-01

Assignment 3

Deadline: 11/25/2025 @ 23:59

[Total Mark for this Assignment is 100]

Student Details:

Name: Bradley Moore **ID: 35737785**

CRN: CSC 575

Instructions:

- You must submit two separate copies **(one Word file and one PDF file)** using the Assignment Template on the Moodle via the allocated folder.
- It is your responsibility to check and make sure that you have uploaded both the correct files.
- Email submission will not be accepted.
- You are advised to make your work clear and well-presented. This includes filling your information on the cover page.
- You must use this template.
- You MUST show all your work, and text must not be converted into an image, unless specified otherwise by the question.
- The work should be your own.
- Use **Times New Roman** font for all your answers.

Question one

(25 Marks)

a. Use the Selection Sort algorithm to sort the following array in ascending order.

[64, 25, 12, 22, 11]

Show the state of the array after each pass (each time the next smallest element is placed in its correct position):

```
import java.util.Arrays;


public class SelectionSort {


    public static void selectionSort(int[] array) {

        for (int i=0; i<array.length; i++){

            System.out.println(Arrays.toString(array));

            int minIndex = i;

            for (int j=i+1; j< array.length; j++) {

                if (array[j] < array[minIndex]) {

                    minIndex = j;

                }

            }

            if (i != minIndex) {

                int temp = array[i];
```

```
            array[i] = array[minIndex];

            array[minIndex] = temp;

        }



    }



  }

  public static void main(String[] args) {

    int[] myArray = {64,25,12,22,11};

    selectionSort(myArray);

  }

}
```

```
PS C:\Users\bradm\OneDrive
_8\bin\java.exe' '-XX:+Show
ionSort'
[64, 25, 12, 22, 11]
[11, 25, 12, 22, 64]
[11, 12, 25, 22, 64]
[11, 12, 22, 25, 64]
[11, 12, 22, 25, 64]
PS C:\Users\bradm\OneDrive
```

b. What is the time complexity of the Selection Sort algorithm?

Because insertion sort does the same number of comparisons regardless of input order it has a time complexity of O(n^2). Even if an array is already sorted in the best-case scenario it still runs through the entire array before being able to determine that the array is sorted.

Question two

25 Marks

Briefly list the advantages and the limitations of the Insertion sort algorithm.

The main advantage of the insertion sort algorithm comes into play when lists are sorted or almost sorted. In these scenarios, insertion sorts time complexity is reduced to O(n) because of the comparative operation to it's previous node allows the algorithm to stay out of an extra loop that would drive the time complexity to O(n^2).

The limitation comes in when working with large dataset that are unsorted. In these cases the algorithm needs to go into another comparative operator which drives time complexity to O(n^2).

Question three

25 Marks

What is the output of the following program:

```java
import java.util.Arrays;
public class ShellSortExample {
    public static void shellSort(int[] arr) {
        int n = arr.length;
        int gap = 1;
        while (gap < n / 3) {
            gap = 3 * gap + 1;
        }
        while (gap > 0) {
            for (int i = gap; i < n; i++) {
                int temp = arr[i];
                int j = i;
                while (j >= gap && arr[j - gap] > temp) {
                    arr[j] = arr[j - gap];
                    j -= gap;
                }
                arr[j] = temp;
            }
            gap = (gap - 1) / 3;
        }
    }
    public static void main(String[] args) {
        int[] numbers = {9, 4, 7, 3, 6, 1, 8, 2, 5};
        System.out.println("Before sorting: " + Arrays.toString(numbers));
        shellSort(numbers);
        System.out.println("After sorting:  " + Arrays.toString(numbers));
    }
}
```

Output:

```
Before sorting: [9, 4, 7, 3, 6, 1, 8, 2, 5]
After Sorting: [1, 2, 3, 4, 5, 6, 7, 8, 9]
PS C:\Users\bradm\OneDrive\Documents\GitHub\ai-masters-workspace\CSC-575-DSA>
```

Notes I took during coding:

//This area sets an efficient "Shell" number based on the length of the array being sorted

// this is called the Knuths Increment Sequence

//for the test array it determines a gap number of 4

//The next part of the code processes the shell sort at the given gap then decrements the gap down by 1 until it reaches 0. At this point the array is sorted.

Question Four

25 Marks

a) Use the Merge Sort algorithm to sort the following array step by step, showing all intermediate splits and merges:

[12, 8, 9, 3, 11, 5, 4]

```
import java.util.Arrays;


public class MergeSort {


    public static int[] merge(int[] array1, int[] array2) {

        int[] combined = new int[array1.length +array2.length];

        int index = 0;

        int i = 0;

        int j = 0;


       while (i < array1.length && j < array2.length) {

            if (array1[i] < array2[j]){

                combined[index] = array1[i];

                System.out.println(combined[index] + " Sorted");

                index++;

                i++;

            }else {

                combined[index] = array2[j];
```

```
            System.out.println(combined[index] + " Sorted");
            index++;
            j++;
        }
    }
    while (i < array1.length){
        combined[index] = array1[i];
        System.out.println(combined[index] + " Sorted");
        index++;
        i++;
    }
    while (j < array2.length){
        combined[index] = array2[j];
        System.out.println(combined[index] + " Sorted");
        index++;
        j++;
    }
return combined;
}
public static int[] mergeSort(int[] array){
    if(array.length ==1){
        System.out.println("Single Array Reached: " + Arrays.toString(array));
```

```
            return array;
        }
        int midIndex = array.length/2;
        int[] left = mergeSort(Arrays.copyOfRange(array, 0, midIndex));
        int[] right = mergeSort(Arrays.copyOfRange(array, midIndex, array.length));
        int [] new_array = merge(left, right);
        System.out.println("Newest Sort: " + Arrays.toString(new_array));
        return new_array;
    }
    public static void main(String[] args) {
        int [] originalArray = {12,8,9,3,11,5,4};


        int [] sortedArray = mergeSort(originalArray);


        System.out.println("Final Outcome: " + Arrays.toString(sortedArray));
    }
}
```

```
_8\bin\java.exe' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\bradm\
ort'
Single Array Reached: [12]
Single Array Reached: [8]
Single Array Reached: [9]
8 Sorted
9 Sorted
Newest Sort: [8, 9]
8 Sorted
9 Sorted
12 Sorted
Newest Sort: [8, 9, 12]
Single Array Reached: [3]
Single Array Reached: [11]
3 Sorted
11 Sorted
Newest Sort: [3, 11]
Single Array Reached: [5]
Single Array Reached: [4]
4 Sorted
5 Sorted
Newest Sort: [4, 5]
3 Sorted
4 Sorted
5 Sorted
11 Sorted
Newest Sort: [3, 4, 5, 11]
3 Sorted
4 Sorted
5 Sorted
8 Sorted
9 Sorted
11 Sorted
12 Sorted
Newest Sort: [3, 4, 5, 8, 9, 11, 12]
Final Outcome: [3, 4, 5, 8, 9, 11, 12]
PS C:\Users\bradm\OneDrive\Documents\GitHub\ai-masters-workspace\CSC-575-DSA>
```

b) What is the time complexity of the Merge Sort algorithm?

When breaking the list in half until we get to single items the time complexity is O(log n)

When combining each item we have to loop through each one once causing O(n) complexity

When these two operations are combined with each other we get O(n log n) time complexity.